

Scan reports and what they mean

Raw tool output is kept in this folder for reproducibility, but per the assignment rules ("do not submit too many scanner screenshots... marks focus on implemented security controls and testing"), this file is the part that should actually be read.

Dependency vulnerabilities (pip-audit)

- **Before fix:** pip_audit_raw run found 10 known CVEs, all in one package: cryptography==43.0.3, pulled in by our own requirements.txt pin of cryptography>=41.0,<44.0. The upper bound was stopping pip from ever installing a fixed release.
- **Fix applied:** requirements.txt now pins cryptography>=44.0.1 (the first release with all of PYSEC-2026-1284 / -2141 / -35 / -3553 / -3554 and GHSA-537c-gmf6-5ccf fixed); the venv here was upgraded to 50.0.1.
- **After fix:** pip_audit_after_fix.txt - "No known vulnerabilities found". pytest (17/17) was re-run after the upgrade to confirm no regression (see docs/test-evidence/pytest_output.txt).
- **OWASP mapping:** A06 - Vulnerable and Outdated Components.

Static analysis (bandit)

Full output: bandit_report.txt (re-run 2026-09-22, bandit 1.9.4). Seven findings remain; none require a further code change:

#	Finding	File	Verdict
1	B608 possible SQL injection	app.py:146 (vulnerable_login)	Intentional. This is the original OWASP-injection teaching route, kept deliberately vulnerable as the "before" side of the Injection control's evidence (see owasp-control-table.md, A03 row). It never establishes a session regardless of query result, so it cannot be used to actually log in as anyone.
2	B608 possible SQL injection	app.py:215 (vulnerable_search)	Intentional, same reasoning - the teaching "before" example; search-secure next to it is the parameterized "after".
3	B104 bind to all interfaces	app.py:325 (app.run(host="0.0.0.0", ...))	Accepted, by design. This is a local lab/classroom server, not a multi-tenant production service; it must bind beyond loopback so the Android emulator (which reaches the host machine as 10.0.2.2, a different address than 127.0.0.1 from the emulator's own network namespace) can connect at all - the same reason the debug mobile build is only

#	Finding	File	Verdict
			ever allowed to talk to that one address (network_security_config.xml).
4	B104 bind to all interfaces	run_https.py:33	Same reasoning as #3 - this is the synthetic local HTTPS host standing in for a real classroom host (see docs/scans/mobile/mitm_pinning_test.txt); it needs to be reachable from the emulator the same way.
5	B404 subprocess import	db.py	Accepted, low risk. Only used once, to run chatter +a on the audit log file (see the PATH-resolution fix below) - no other use of subprocess in the codebase.
6/7	B105 "hardcoded password"	templates.py (PASSWORD_RESET_*_BODY)	False positive. Bandit's heuristic triggers on the variable <i>name</i> containing "password"; the value is an HTML template string, not a credential. Correction made this pass: this file previously carried a # nosec B105 comment on the line <i>before</i> each flagged assignment - Bandit matches # nosec against the exact physical line number it reports (the assignment line itself), so a comment on the preceding line has no effect at all. It was silently not suppressing anything, and the report still listed both as active findings even before this fix, contradicting what the comment implied. Since these are multi-line triple-quoted string assignments, an inline trailing # nosec on the same line isn't safe either - anything after "" on that line becomes part of the string's literal content, which would inject stray text into the rendered HTML page. The comments were corrected to accurately describe the reasoning without falsely claiming suppression; the findings remain visible in the report by design, with this explanation as the record of review.

Note on products.py/users_admin.py: their UPDATE-statement f-strings (hardcoded "<column> = ?" fragments from a fixed allowlist, never user input, with every actual value still bound as a ? parameter) do **not** appear in this fresh run at all - the existing inline # nsec B608 comments on those exact lines are correctly formed and bandit's own "potential issues skipped due to specifically being disabled" counter confirms 3 suppressions took effect (db.py:201, products.py:123, users_admin.py:66).

Real fixes applied as a result of scans (this pass and earlier):

- db.py's chatr call previously invoked the bare command name (["chatr", "+a", path]), which Bandit flagged as B607 (partial executable path - vulnerable to PATH manipulation). It now resolves the absolute path with shutil.which() first and skips the OS-level hardening step with a warning if chatr isn't found, instead of trusting \$PATH. Confirmed fixed: B607 no longer appears in bandit_report.txt.
- The templates.py B105 nsec-placement bug described in row 6/7 above.

OWASP mapping: A03 - Injection (rows 1/2, the deliberate teaching example; the products.py/users_admin.py pattern noted above is the confirmed-safe counterpart), A05 - Security Misconfiguration (rows 3/4, and the PATH-resolution fix).

SBOM

- docs/sbom/backend-sbom.json - real CycloneDX SBOM generated with pip-audit --format=cyclonedx-json against the final requirements.txt (32 components: every direct *and* transitive dependency actually installed in the venv, not just what's listed in requirements.txt; 0 known vulnerabilities, re-verified 2026-09-22).
- docs/sbom/mobile-sbom.json - real CycloneDX SBOM, regenerated 2026-09-22 with the CycloneDX Gradle plugin (./gradlew cyclonedxBom, added to app/build.gradle.kts) once an Android SDK became available in this environment. **282 components** - the full resolved dependency graph (all direct + transitive AndroidX/Kotlin/Google libraries), not just the ~19 direct entries in gradle/libs.versions.toml. This replaces a prior hand-authored version that had gone stale: it still listed net.openid:appauth, a dependency removed when mobile Google Sign-In was migrated to Credential Manager (see GoogleAuthConfig.kt), and was missing androidx.credentials/googleid entirely. Regenerate any time with cd mobile/Product_Search_Project && ./gradlew cyclonedxBom, output at app/build/reports/cyclonedx/bom.json.

Mobile scans

All of the following now have real, reproducible output in docs/scans/mobile/, generated 2026-09-22 once an Android SDK/emulator became available in this environment (they were previously blocked and left as a manual checklist - see docs/mobile-security-checklist.md):

- apk_secret_scan.txt - release APK unzipped and grepped for client_secret/GOCSPX-/API keys/PEM private keys. Clean; the only matches are Kotlin property *names* (password=, new_password=), not values.
- apk_debuggable_check.txt - confirms android:debuggable is absent from the release manifest and allowBackup=false; also confirms R8 actually renamed the app's own classes (0 matches for ProductRepository/AppViewModel/etc. in the dex, vs. the one manifest-declared MainActivity that must keep its real name).
- local_storage_check.txt - TokenStore's EncryptedSharedPreferences file has both encrypted keys *and* values; no plaintext JWT, no stray SQLite cache.
- mitm_pinning_test.txt - the full MITM/TLS-pinning demonstration against a synthetic local HTTPS host (see below), including the negative test (substituted certificate rejected with SSLHandshakeException: Trust anchor for certification path not found, observed against the real R8-obfuscated release build) and the recovery test

(works again once the legitimate certificate is restored, proving the rejection was specific to the substitution, not a general break).

A decompiled-source secret scan (apktool/jadx) was not run - those tools aren't installed in this environment - but apk_secret_scan.txt's strings-based scan reads the same underlying dex string table a decompiler would, so it is not a materially weaker check for this purpose, only a less nicely formatted one.

No real classroom host was provided for this assignment. Per Rule 3 (synthetic test data only), the release build's API_BASE_URL and network_security_config.xml's pinned domain/certificate point at a locally-generated, throwaway self-signed CA instead (backend/run_https.py, backend/certs/, gitignored) - see mitm_pinning_test.txt for the full setup and both the positive and negative test results.